

Харківський національний університет імені В.Н. Каразіна

Кафедра: ІНТЕЛЕКТУАЛЬНИХ ПРОГРАМНИХ СИСТЕМ І ТЕХНОЛОГІЙ

Дисципліна: МЕТОДИ ТА ІНСТРУМЕНТАЛЬНІ ЗАСОБИ ТЕСТУВАННЯ
ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Лектор: А.А. Мартиненко

Викладачі, які проводять п/р: А.А. Мартиненко

ПРАКТИЧНА РОБОТА №4

Тема: Автоматизоване тестування веб-інтерфейсу за допомогою Selenium

Харків 2026

Мета

1. Опанувати базові навички автоматизованого тестування веб-інтерфейсів, використовуючи Selenium WebDriver.
2. Закріпити техніки тест-дизайну (еквівалентне розбиття, аналіз граничних значень, попарне тестування) на практичних прикладах.
3. Реалізувати Page Object Model (POM) у тестовому фреймворку для підвищення підтримуваності та масштабованості автотестів.

1. Теоретична частина

Основи Selenium WebDriver

Selenium – це набір інструментів для автоматизованого тестування веб-додатків, який дозволяє керувати браузером програмно. Selenium WebDriver є основним компонентом, що підтримує крос-браузерне тестування – тобто виконання тестів у різних браузерах (Chrome, Firefox, Safari, Edge тощо) під управлінням різних мов програмування (Java, Python, C#, JavaScript та ін.) . WebDriver імітує дії користувача (натискання, введення тексту, навігацію) у реальному браузері, дозволяючи перевірити, чи веб-застосунок працює належним чином у реальних умовах.

Основні компоненти Selenium WebDriver:

- WebDriver – інтерфейс (та його реалізації), через який відбувається керування браузером. Для кожного браузера є свій драйвер – окремий виконуваний файл або клас. Наприклад, для Chrome – ChromeDriver, Firefox – FirefoxDriver (GeckoDriver), Edge – EdgeDriver тощо . Програма створює об'єкт відповідного драйвера (наприклад, `new ChromeDriver()`), після чого через його методи виконує дії в браузері.

- WebElement – клас, що представляє HTML-елемент на веб-сторінці. Після отримання елемента, наприклад через метод пошуку, об'єкт WebElement дозволяє виконувати дії над цим елементом: клікати, вводити текст, читати текст тощо . WebElement фактично є об'єктом, який містить елемент DOM-дерева та методи для взаємодії з ним.

- By – це клас (абстракція), що надає локатори для пошуку елементів на сторінці. Найпоширеніші способи пошуку:

- By.id,
- By.name,

- `By.className`,
- `By.cssSelector`,
- `By.xpath`,
- `By.linkText`,
- `By.partialLinkText`,
- `By.tagName` .

Наприклад, `driver.findElement(By.id("login"))` знайде елемент з атрибутом `id="login"`. Метод `findElement` класу `WebDriver` приймає об'єкт `By` і повертає перший знайдений `WebElement` (або кидає виняток, якщо не знайдено), а `findElements` повертає список усіх відповідних елементів.

Можливості `Selenium WebDriver` включають навігацію по сторінках, імітацію дій користувача та перевірку стану сторінки. Наприклад, `WebDriver` дозволяє:

- Відкрити веб-сторінку за URL (`driver.get("https://example.com")`).
- Отримати назву сторінки (`driver.getTitle()`) чи іншу інформацію (URL, HTML-код, куки тощо).
- Знайти елемент на сторінці (`findElement(By...)` або `findElements(By...)`) і виконати дію: клік на кнопку або посилання (`element.click()`), введення тексту в поле (`element.sendKeys("текст")`), очищення поля (`element.clear()`), вибір значення зі списку тощо .
- Керувати вікном браузера – мінімізувати/розгорнути, перемикаючи вкладки, виконувати навігацію назад/вперед, перемикатися в інші фрейми чи вікна.

- Обробляти діалоги (JavaScript alert/confirm), виконувати JavaScript, завантажувати файли і багато іншого.

Selenium WebDriver базується на архітектурі клієнт-сервер: тестовий скрипт (клієнт) надсилає команди WebDriver, які через спеціальний браузерний драйвер керують реальним браузером. На схемі нижче показано спрощену архітектуру WebDriver: клієнтські бібліотеки Selenium надсилають команди через протокол (в Selenium 3 – JSON Wire, в Selenium 4 – W3C WebDriver) до браузерного драйвера, а той – безпосередньо до браузера, який виконує дії і повертає результати назад клієнту. Це дозволяє автоматизовано і прозоро керувати браузерами різних типів.

Написання першого тесту

Розглянемо простий приклад автоматизованого тесту: відкриття веб-сайту і перевірка його заголовка. Цей тест переконається, що ми правильно взаємодіємо з браузером через Selenium. Для початку створимо клас з методом `main` (або тестовий метод у фреймворку JUnit/TestNG) і виконаємо там кроки:

1. Запуск браузера і перехід на сторінку. Спочатку ініціалізуємо WebDriver (наприклад, ChromeDriver) і відкриємо потрібний URL.
2. Отримання заголовка сторінки. Скористаємося методом `getTitle()` і збережемо або одразу порівняємо результат.
3. Перевірка заголовка. Порівняємо отриманий заголовок із очікуваним значенням. Якщо збігається – тест проходить, якщо ні – фіксуємо помилку.

4. Закриття браузера. Після перевірки закриємо браузер (quit()).

Нижче наведено код, що реалізує ці кроки (в якості сайту для тесту використано Google):

```
System.setProperty("webdriver.chrome.driver", "C:\\drivers\\chromedriver.exe");
WebDriver driver = new ChromeDriver();
driver.get("https://www.google.com/"); // 1. Відкрити сторінку Google
String title = driver.getTitle(); // 2. Отримати заголовок
if ("Google".equals(title)) { // 3. Перевірити заголовок
    System.out.println("Тест пройдено: заголовок - " + title);
} else {
    System.out.println("Тест НЕ пройдено: очікувався 'Google', а отримано '" + title + "'");
}
driver.quit(); // 4. Закрити браузер
```

Page Object Model (POM) – модель сторінок

У міру того, як тестовий проєкт зростає, виникає потреба зробити тести більш підтримуваними і читабельними. Page Object Model (POM) – це поширений шаблон проєктування для автоматизованих UI-тестів, що допомагає організувати код і зменшити дублювання. Ідея POM полягає в тому, щоб представити кожну веб-сторінку (або значущу частину інтерфейсу) у вигляді окремого класу – так званого Page Object. Цей клас містить:

- Елементи сторінки (поля класу типу WebElement або локатори By), які відповідають важливим елементам UI на цій сторінці.
- Методи-взаємодії з цією сторінкою, що реалізують типові дії користувача на ній (наприклад, метод логіну, метод пошуку, натискання кнопки тощо).

Таким чином, тестові сценарії можуть створювати об'єкти відповідних Page Object класів і викликати їхні методи, не займаючись безпосередньо пошуком елементів – вся логіка взаємодії інкапсульована в класах сторінок. Це має кілька суттєвих переваг:

- Покращена підтримуваність. Якщо змінюється верстка сторінки або локатори елементів, виправлення вносяться лише в одному місці – у відповідному Page Object класі, а не в кожному тесті.
- Зменшення дублювання коду. Загальні для багатьох тестів операції (наприклад, логін) реалізуються один раз у Page Object і можуть багаторазово використовуватися в різних тестах.
- Краща читабельність тестів. Тестові методи виглядають як послідовність логічних дій (напр. `loginPage.login(username, password); dashboardPage.logout();`), без деталей реалізації цих дій, що робить тести більш зрозумілими.
- Інкапсуляція. Всі деталі роботи з елементами UI сховані всередині Page Object. Тести взаємодіють лише з високорівневими методами, що зменшує вплив змін в UI на сам тест.
- Можливість повторного використання. Одні й ті самі об'єкти сторінок можуть застосовуватися в різних тестах, а методи – комбінуватися по-іншому для різних сценаріїв. Це полегшує масштабування тестового набору.

ПОМ зараз є де-факто стандартом написання UI-тестів. Згідно з цим підходом, структура проекту зазвичай розділяється на дві частини: класи сторінок (Page Objects) і класи тестів. Класи сторінок розташовуються в окремому пакеті (наприклад, `pages`), класи тестів – у пакеті `tests` або `testCases`. Така структура розділяє логіку UI та самі тести, що також сприяє чистоті архітектури тестового проекту.

Приклад

Розглянемо приклад тесту, написаного з використанням Page Object Model на Java. Припустимо, є демо-сайт з книжковим магазином, де на головній сторінці є кнопка логіну, сторінка логіну з полями введення і кнопкою входу, та сторінка профілю (dashboard) після успішного входу. Ми створимо POM-класи для кожної з цих сторінок: HomePage, LoginPage та DashboardPage, а потім напишемо тест, який використовує ці класи.

```
package pages;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {
    WebDriver driver;
    // Конструктор – отримує WebDriver і зберігає для подальшого використання
    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }
    // Локатори полів і кнопки
    private By usernameField = By.id("userName");
    private By passwordField = By.id("password");
    private By loginBtn = By.id("login");
    // Методи вводу логіну, паролю та натискання кнопки
    public void enterUsername(String user) {
        driver.findElement(usernameField).sendKeys(user);
    }
    public void enterPassword(String pass) {
        driver.findElement(passwordField).sendKeys(pass);
    }
    public void clickLogin() {
        driver.findElement(loginBtn).click();
    }
}
```

У цьому класі конструктор приймає WebDriver, щоб дозволити методам надсилати команди браузеру. Локатори визначені як приватні поля By, а публічні методи enterUsername, enterPassword, clickLogin реалізують взаємодію з відповідними елементами. Подібним чином можна створити клас HomePage (з методом clickLogin() для натискання кнопки входу на головній сторінці) та DashboardPage (з, наприклад, методом getHeading() для отримання заголовка сторінки профілю і clickLogout() для виходу).

Тестовий клас може виглядати так (без використання фреймворку, просто з методом main для наочності):

```
package testCases;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import pages.HomePage;
import pages.LoginPage;
import pages.Dashboard;

public class LoginTest {
    public static void main(String[] args) throws InterruptedException {
        System.setProperty("webdriver.chrome.driver", "C:\\path\\to\\chromedriver.exe");
        WebDriver driver = new ChromeDriver();
        driver.get("https://www.demoma.com/books");

        // Ініціалізація об'єктів сторінок
        HomePage home = new HomePage(driver);
        LoginPage login = new LoginPage(driver);
        Dashboard dashboard = new Dashboard(driver);

        // Кроки тестового сценарію
        home.clickLogin(); // натиснути "Login" на головній
        login.enterUsername("DemoUser"); // ввести ім'я
        login.enterPassword("DemoPassword"); // ввести пароль
        login.clickLogin(); // натиснути кнопку входу
        Thread.sleep(3000); // Явна затримка для завантаження сторінки (небажано у реальних тестах)
        String heading = dashboard.getHeading(); // отримати заголовок профілю
        System.out.println("Заголовок сторінки: " + heading);
        dashboard.clickLogout(); // натиснути "Logout"
        driver.quit();
    }
}
```

Зверніть увагу: після виконання будь-яких дій варто переконатися, що вони дійсно відбулися. Наприклад, після кліку по чекбоксу – перевірити, що він позначений:

```
assertTrue(checkbox.isSelected());
```

Після відправки форми – зачекати, що завантажилась нова сторінка або з'явилося підтвердження. Selenium надає методи отримання стану елемента: `isDisplayed()`, `isEnabled()`, `isSelected()`, а також можливість читати текст: `getText()` та атрибути: `getAttribute(String)` – їх можна використовувати для валідації результатів дій у тесті.

2. Завдання

Інструменти:

- Мова програмування на вибір (наприклад, Java + JUnit/TestNG, або Python + PyTest, тощо).
- Selenium WebDriver відповідної версії.
- Веб-драйвер браузера (ChromeDriver, GeckoDriver тощо) для обраного браузера (Google Chrome, Mozilla Firefox та ін.).
- Будь-який тестовий фреймворк для організації запуску тестів і перевірок (assertions).
- Тестовий веб-сайт.
- Інструменти для аналізу комбінацій

Завдання 1: Тестування форми входу

Реалізуйте простий автоматизований тест для форми авторизації на веб-сайті. В якості тестового майданчика можна використати демо-сторінку входу (наприклад, The Internet (Herokuapp) Login Page – <https://the-internet.herokuapp.com/login>). Необхідно перевірити поведінку форми при коректних та некоректних даних входу. Використайте методи аналізу граничних значень для визначення тестових даних.

Завдання 2: Тестування форми реєстрації

Розробіть автоматизований набір тестів для перевірки веб-форми реєстрації, яка містить кілька типів полів вводу. В якості прикладу можна використати демонстраційну форму “Student Registration Form” на сайті DemoQA – <https://demoqa.com/automation-practice-form> (англомовний сайт) або знайти аналогічну тестову форму.

Форму необхідно заповнювати різними наборами даних, перевіряючи валідацію полів та поведінку інтерфейсу. Page Object Model слід використати для структурування взаємодії з формою (наприклад, створити клас FormPage).

Еквівалентне розбиття та граничні значення слід застосувати для кожного поля, а при комбінуванні кількох параметрів – задуматися про ефективність тестів (можна частково застосувати принципи попарного тестування для комбінування декількох ввідних даних).

Завдання 3: Тестування пошуку та фільтрації на e-commerce сайті

Реалізуйте комплексне автоматизоване тестування функціоналу пошуку товарів та застосування фільтрів на веб-сайті електронної комерції. В якості платформи оберіть реальний сайт.

Сайт має містити пошук по назві товару, по категорії товару, по частковій назві товару, тощо. Сайт також повинен надавати змогу користувачеві фільтрувати за брендом, ціною, кольором, тощо.

Завдання включає кілька етапів: виконання пошукових запитів, навігація на сторінку результатів пошуку для перевірки коректності пошуку та застосування фільтрів (наприклад, за категорією, брендом, ціною).

Для структурування тестів створіть Page Object'и для головної сторінки/панелі пошуку та для сторінки результатів (і панелі фільтрів).

Техніки тест-дизайну: використайте еквівалентне розбиття, тестування таблиць рішень і граничні значення для формування набору пошукових запитів, а для перевірки різних комбінацій фільтрів застосуйте попарне тестування (щоб покрити всі пари значень фільтрів без тестування кожної можливої комбінації).

Завдання 4*: Реалізуйте можливість створення скріншоту і додавання його до звіту в разі помилки (багу) яку знайде тест.

3. Оформлення звіту

- 1) До звіту додати таблиці дизайну тестів за допомогою еквівалентного розбиття, граничних значень і попарного тестування.
- 2) Код основних елементів та тестів.
- 3) Звіт після тестування (кожен тест має бути позначений як Passed/Failed).
- 4) Мати змогу продемонструвати роботу тестів.

4. Контрольні запитання

- 1) Що таке Selenium WebDriver та в чому полягає його роль у автоматизації тестування веб-інтерфейсів?
- 2) Яка відмінність між неявним (Implicit) та явним (Explicit) очікуванням в Selenium? У яких випадках бажано використовувати кожне з них?
- 3) Яку роль у побудові автотестів відіграє Page Object Model (POM)? Які переваги він дає порівняно зі “звичайним” написанням тестів?
- 4) Які існують основні методи пошуку елементів (By) у Selenium, і в чому перевага використання того чи іншого локатора?
- 5) Чим відрізняються методи findElement та findElements у Selenium WebDriver і як вони обробляють відсутність елемента на сторінці?

6) Що таке еквівалентне розбиття та аналіз граничних значень у тест-дизайні? Наведіть приклад для текстового поля з обмеженням довжини.

7) Як можна застосувати попарне тестування (pairwise testing) при перевірці декількох фільтрів або параметрів введення?

5. Список додаткових джерел інформації

- <https://www.selenium.dev/documentation/>
- <https://dou.ua/forums/topic/44882/>
- Грем Д., ван Вінендал Е., Еванс І., Блек Р. Foundations of Software Testing: ISTQB Certification. Лондон: Cengage Learning EMEA, 2008.